

Sistemas Distribuidos y Paralelos

Ingeniería en Computación



Modelo de programación híbrido

Universidad Nacional de La Plata



Facultad de Informática





1

Programación híbrida

2

MPI + Pthreads

3

MPI + OpenMP

4

Modularidad

Modularidad MPI+Pthreads. Modularidad MPI+OpenMP. Compilación y dificultades en el uso de modularidad.



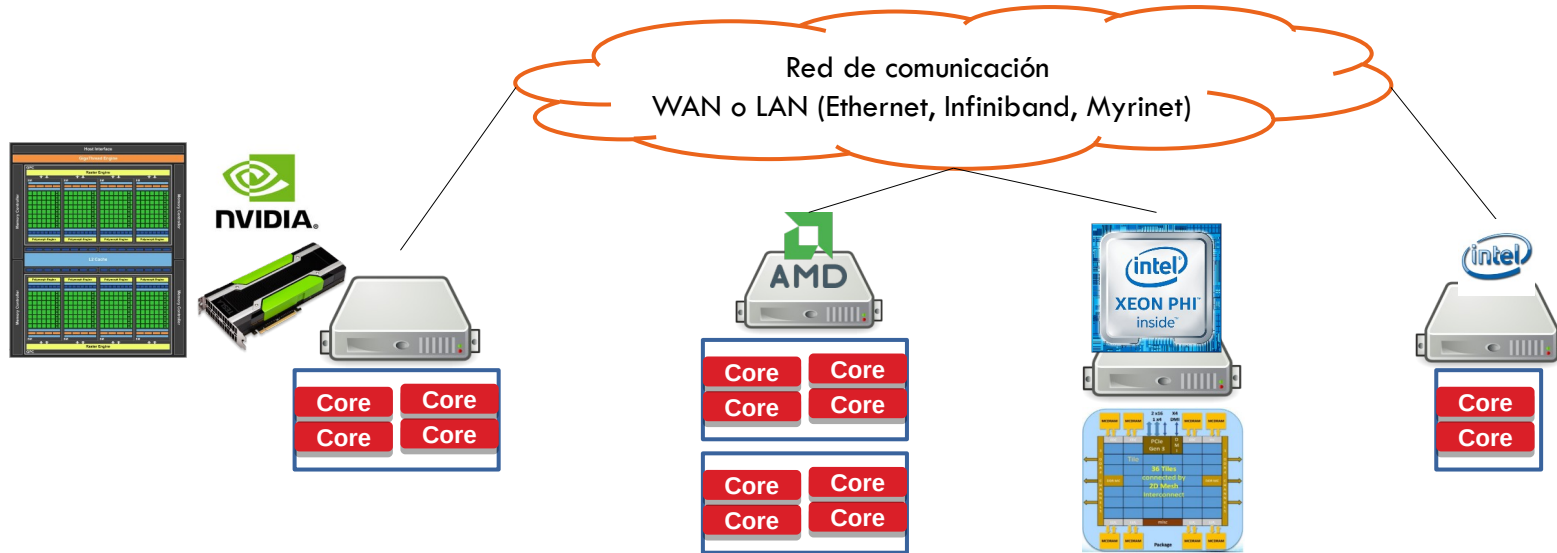
1

Programación híbrida

Modelo de programación híbrido

4

- Actualmente, contamos con una gran diversidad de arquitecturas de cómputo.
- Arquitecturas de memoria compartida y memoria distribuida combinadas formando una **arquitectura híbrida** con mayor potencia de cómputo.



Modelo de programación híbrido

5

- Es necesario combinar distintas herramientas:

- MPI+Pthreads
- MPI+OpenMP
- Otros:
 - MPI+Cuda, MPI+OpenMP+Cuda, OpenMP+Cuda etc.

- Varias razones para elegir una u otra combinación:

- Herramientas de memoria compartida (Pthreads y OpenMP) no pueden usarse sobre clusters.
- MPI sobre multicores puede consumir mas memoria que Pthreads y OpenMP.
- Uso de herramientas específicas: sobre GPU sólo podemos usar CUDA u OpenCL.

Mayor complejidad a la hora de programar, compilar y ejecutar.



Modelo de memoria Híbrido

Modelos de programación paralela – Modelos de arquitectura paralela

6

- El modelo de programación híbrido conveniente sólo en arquitecturas híbridas.

		Software		
		Memoria Compartida (Ej: OpenMP)	Memoria Distribuida (Ej: MPI)	Híbrido (EJ: MPI + OpenMP)
Hardware	Memoria Compartida (Ej: multicore)	Trivial (Ej: OpenMP sobre Multicore)	Posible (Ej: MPI sobre multicore)	Posible (extraño) (Ej: MPI + OpenMP sobre multicore)
	Memoria Distribuida (Ej: cluster moncore ¹)	(NO ADECUADO) Single System Image(SSI) Overhead	Trivial (Ej: MPI sobre cluster)	Posible (muy extraño) (Ej: MPI + OpenMP sobre cluster moncore)
	Híbrido (Ej: cluster multicore)	(NO ADECUADO) Single System Image(SSI) Overhead	Posible (Ej: MPI sobre cluster multicore)	Trivial (Ej: MPI + OpenMP sobre cluster multicore)

¹Actualmente, los clusters moncore cayeron en desuso pero se mencionan para ejemplificar.

Modelo de programación híbrido

Alternativa de inicialización en MPI

7

- **MPI + hilos** requiere inicializar el entorno de forma ***thread safety*** con la función :

```
int MPI_Init_thread(int *argc, char ***argv, int required, int *provided)
```

required: Nivel deseado de soporte para hilos. Valores posibles:

MPI_THREAD_SINGLE: El proceso se ejecutará con un único hilo.

MPI_THREAD_FUNNELED: sólo el hilo que invocó a `MPI_Init_thread` puede llamar funciones MPI.

MPI_THREAD_SERIALIZED: varios hilos pueden llamar funciones MPI, pero no concurrentemente.

MPI_THREAD_MULTIPLE: varios hilos pueden llamar funciones MPI, sin restricciones.

provided: puntero a un valor de retorno que indica el nivel de soporte para hilos otorgado

Puede ser igual o menor al nivel solicitado en `required`.

Es responsabilidad del programador verificar que el soporte disponible sea el requerido.



Podemos tener una aplicación híbrida con **`MPI_Init`**
No garantiza un entorno seguro al invocar funciones MPI

Modelo de programación híbrido

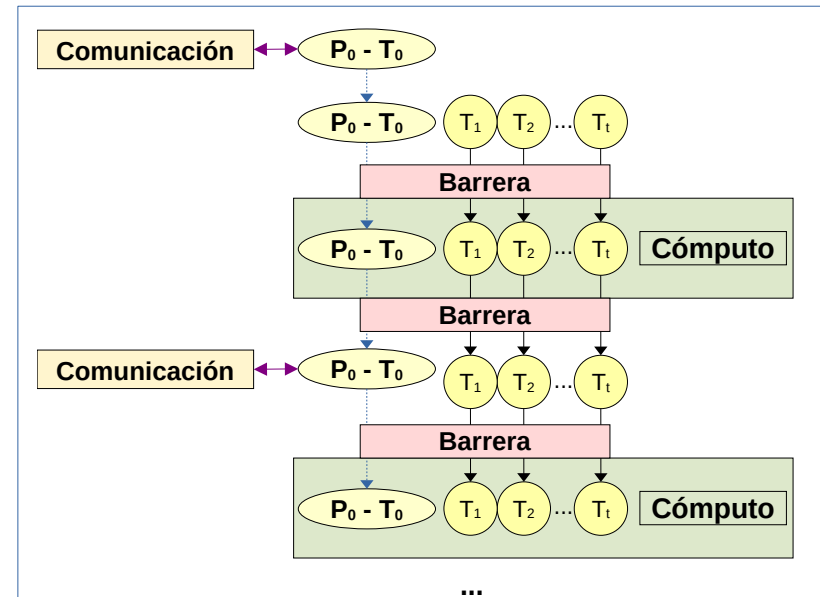
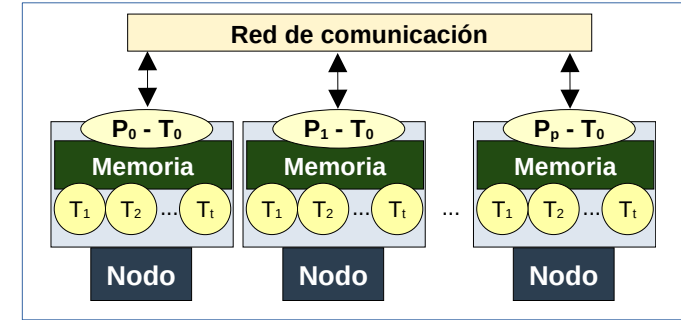
Estrategia general MPI+Pthreads y MPI+OpenMP

8

- Estrategia habitual MPI+Pthreads y MPI+OpenMP es:

- Crear un proceso MPI por nodo del cluster.
- Cada proceso crea varios hilos en el nodo.
- **Recomendable: Ejecución en etapas**

- El proceso realiza comunicación *inter-nodo*.
 - *Eventualmente, también trabaja.*
- Los hilos sólo cómputo.
 - **MPI no garantizan soporte MPI_THREAD_MULTIPLE eficiente.**





2

MPI + Pthreads

MPI+Pthreads

Código ejemplo

10

mpi_pthreads.c

```
#include <mpi.h>
#include <pthread.h>
#define T ...
int rank;

void* f(void *arg){
int tid = *(int*)arg;
...
pthread_exit(NULL);
}

int main(int argc, char* argv[]){
int provided;
pthread_t h[T];
MPI_Init_thread(&argc, &argv,
MPI_THREAD_FUNNELED, &provided);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

for(int tid=1; tid<T-1; tid++){
pthread_create(&h[tid], NULL, &f, (void*) tid);
}
f((void *) 0); // Hilo principal también trabaja
for(int tid=0; tid<T; tid++){
pthread_join(h[tid], NULL);
}

MPI_Finalize();
return(0);
}
```

Ejecución en etapas

```
#include <mpi.h>
#include <pthread.h>
#define T ...
int rank;

pthread_barrier_t barrera;

void* f(void *arg){
int tid = *(int*)arg;

if ( tid == 0 ) { // Comunica }
pthread_barrier_wait(&barrera); //Sincronizan

... // Computan
pthread_barrier_wait(&barrera); //Sincronizan
if ( tid == 0 ) { // Comunica }
pthread_barrier_wait(&barrera); //Sincronizan
...
pthread_exit(NULL);
}
...
```

```
mpi_pthreads.c
```

Compilar: `mpicc -lpthread -o mpi_pthreads mpi_pthreads.c`

Ejecutar: `mpirun --bind-to none -np 2 -machinefile maquinas mpi_pthreads`

BUG de MPI!!!

Opción para que no ejecute todos los hilos en el mismo core!!!



Archivo "maquinas"

```
Maquina1 slots=1  
Maquina2 slots=1  
...
```



3

MPI + OpenMP

MPI+OpenMP

Código ejemplo

13

```
#include <mpi.h>
#include <omp.h>

int main(int argc, char* argv[]){
    int rank;
    int provided;

    MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Comunica
    #pragma omp parallel
    {
        ... // Computan
    } // Sincronizan (Barrera implícita)
    // Comunica
    #pragma omp parallel
    {
        ... // Computan
    } // Sincronizan (Barrera implícita)
    ...
    MPI_Finalize();
    return(0);
}
```

mpi_omp.c

MPI+OpenMP

Compilación y ejecución

14

```
mpi_omp.c
```

Compilar: `mpicc -fopenmp -o mpi_omp mpi_omp.c`

Ejecutar: `mpirun --bind-to none -np 2 -machinefile maquinas mpi_omp`

BUG de MPI!!!

Opción para que no ejecute todos los hilos en el mismo core!!!



Archivo “maquinas”

```
Maquina1 slots=1  
Maquina2 slots=1  
...
```

Modularidad

Modularidad MPI+Pthreads

Modularidad MPI+OpenMP

Compilación y dificultades en el uso de modularidad



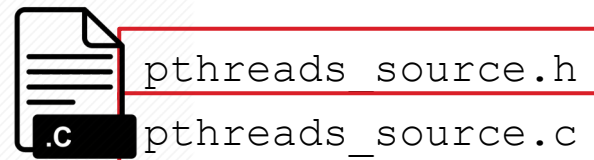
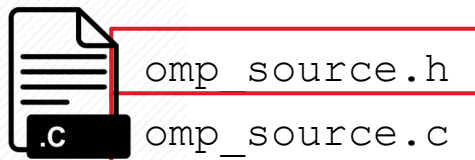
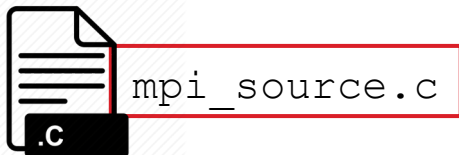
Modularidad

16

- Recomendable modularizar: código de cada herramienta por separado.



- Responsabilidades claramente definidas por módulo.
- Facilidad para depurar, mayor modularidad y portabilidad.



- En ocasiones no contamos con el código fuente.

- Dos opciones de compilación y *link* edición:



- 1) Un mismo compilador para compilar todos los módulos
- 2) Compiladores diferentes para cada módulo generando código objeto (.o) y luego “*linkear*” el código objeto con alguno de los compiladores.

Modularidad

Modularidad MPI+Pthreads

Modularidad MPI+OpenMP

Compilación y dificultades en el uso de modularidad



Modularidad

MPI+Pthreads – Código ejemplo

18

```
#ifndef __PTHREADS_FUNCTIONS__
#define __PTHREADS_FUNCTIONS__
extern void pthreads_function(par_t pars);
#endif
```

pthread_source.h

```
#include <mpi.h>
#include <pthread_source.h>
```

mpi_source.c

```
static void mpi_function(int rank){
    // Comunica
    pthreads_function(par_t pars);
    // Comunica
}

int main(int argc, char* argv[]){
    int rank;
    int provided;

    MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    mpi_function(rank);

    MPI_Finalize();
    return(0);
}
```

```
#include <pthread.h>
#define T ...
par_t shared_pars;

void* f(void *arg){
    int tid = *(int*)arg;

    ... // Computan

    pthread_exit(NULL);
}

void pthreads_function(par_t pars){
    pthread_t h[T];
    shared_pars = pars;

    for(int tid=0;tid<T;tid++){
        pthread_create(&h[tid], NULL, &f, (void*)tid);
    }

    for(int tid=0;tid<T;tid++) // Sincronizan
        pthread_join(h[tid], NULL);
}
```

pthread_source.c

Modularidad

MPI-Pthreads – Compilación

19



- **Opción 1:** Compilación unificada con el compilador de MPI:

```
mpicc -lpthread -o mpi_pthreads mpi_source.c pthreads_source.c
```



- **Opción 2:** Compilación por módulo en 3 pasos:

- **Compilar sólo MPI:**

```
mpicc -c mpi_source.c -o mpi_source.o
```

- **Compilar sólo Pthreads:**

```
gcc -lpthread -c pthreads_source.c -o pthreads_source.o
```

- **Linkear con el compilador de MPI:**

```
mpicc -o mpi_pthreads mpi_source.o pthreads_source.o
```

Modularidad

Modularidad MPI+Pthreads

Modularidad MPI+OpenMP

Compilación y dificultades en el uso de modularidad



Modularidad

MPI-OpenMP – Código ejemplo

21

```
#include <mpi.h>
#include <omp_source.h>
```

mpi_source.c

```
static void omp_function(int rank){
    // Comunica
    omp_function(par_t pars);
    // Comunica
}
```

```
int main(int argc, char* argv[]){
    int rank;
    int provided;
```

```
    MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    mpi_function(rank);
```

```
    MPI_Finalize();
    return(0);
}
```

```
#ifndef __OMP_FUNCTIONS__
#define __OMP_FUNCTIONS__
    extern void omp_function(par_t pars);
#endif
```

omp_source.h

```
#include <omp.h>
```

omp_source.c

```
void omp_function(par_t pars){
    par_t shared_pars = pars;

    #pragma omp parallel
    {
        ... // Computan
    } // Sincronizan
}
```

Modularidad

MPI+OpenMP – Compilación

22



- **Opción 1:** Compilación unificada con el compilador de MPI:

```
mpicc -fopenmp -o mpi_omp mpi_source.c omp_source.c
```



- **Opción 2:** Compilación por módulo en 3 pasos:

- **Compilar sólo MPI:**

```
mpicc -c mpi_source.c -o mpi_source.o
```

- **Compilar sólo OpenMP:**

```
gcc -fopenmp -c omp_source.c -o omp_source.o
```

- **“Linkear” con el compilador de MPI:**

```
mpicc -lgomp -o mpi_omp mpi_source.o omp_source.o
```

Modularidad

Modularidad MPI+Pthreads

Modularidad MPI+OpenMP

Compilación y dificultades en el uso de modularidad



- Compilación: cualquier opción es válida.
- Algunos algoritmos dificultan la tarea de mantener código separado:
 - Algoritmos por etapas con comunicación en cada una de ellas:
 - MPI+Pthreads: crear y destruir hilos
 - Posible solución: gestionar un *pool* de *threads*.
 - MPI+OpenMP: gestionado por Runtime System.
 - Hilos que comunican: evitarlo siempre que se pueda.



Próxima clase

Capítulo 9

Métricas



